

WYŻSZA SZKOŁA
INFORMATYKI i ZARZĄDZANIA
z siedzibą w Rzeszowie

Kolegium Informatyki Stosowanej

Projekt z przedmiotu "Szkolenie Techniczne 2"

Serwis hotelowy

Prowadzący:
mgr inż. Łukasz Piechocki

Grupa:
4IID-P/2023-SPL03

Autor:
Viktor Pylypenko

Numer albumu:
w68166

Rzeszów, 25 czerwca 2025

Spis treści

Opis wybranego stosu technologicznego 3
Instrukcja uruchamiania aplikacji 4
Opis struktury bazy danych 5
Tabele systemowe ASP.NET Identity 5
Tabele systemowe Entity Framework 6
Modele biznesowe aplikacji 6
Historia zmian encji 7
Diagram przypadków użycia 8
Opis kluczowych elementów backendu 9
Kontrolery 9
Serwisy aplikacyjne 10
System filtrowania 11
Repozytoria 11
Obsługa zdarzeń 12
Walidacja danych 12
Opis przypadków testowych 13
Literatura 15

Opis wybranego stosu technologicznego

Do budowy aplikacji webowej zastosowano platformę .NET z językiem C#, opartą na architekturze warstwowej. W warstwie prezentacji wykorzystano ASP.NET Core MVC wraz z Razor Pages, co pozwala na wygodne tworzenie stron renderowanych po stronie serwera. Dzięki Razor możliwe jest łączenie danych z elementami HTML w sposób przejrzysty i intuicyjny. Tag helpery ułatwiają obsługę formularzy, walidację oraz dynamiczne przypisywanie atrybutów bez nadmiernego powtarzania kodu.

Interfejs użytkownika uzupełniono o jQuery, służące do reagowania na zdarzenia i dodawania interaktywności na stronie, np. przechwytywania kliknięć, uzupełniania pól czy ukrywania sekcji. Do warstwy wizualnej dołączono Bootstrap, który pozwala na szybkie projektowanie responsywnego i estetycznego układu. Stylizacja oraz układ elementów nie wymagają pisania własnych arkuszy CSS od podstaw, co przyspiesza prace nad wyglądem i zwiększa spójność interfejsu.

Dane aplikacji przechowywane są w relacyjnej bazie SQL Server, dobranej ze względu na pełną zgodność z ekosystemem .NET oraz dobrą wydajność w pracy z większymi strukturami danych. Wspierane są mechanizmy transakcji, widoki, klucze obce i zaawansowane zapytania, co odpowiada wymaganiom aplikacji biznesowej.

Komunikacja z bazą danych odbywa się z użyciem Entity Framework Core, działającego w podejściu Code First. Model danych definiowany jest w kodzie C#, a migracje bazy generowane automatycznie. Pozwala to utrzymać spójność struktury danych z logiką domenową oraz przyspiesza rozwój i modyfikacje schematu bazy bez konieczności pisania zapytań SQL.

Całość rozwiązania oparta jest na zasadach czystej architektury. Kod aplikacji podzielony jest na niezależne warstwy: domenę, logikę aplikacyjną, infrastrukturę oraz warstwę prezentacji. Taki układ zwiększa czytelność kodu, ułatwia testowanie i pozwala na elastyczną rozbudowę projektu w przyszłości — bez ryzyka niepożądanych zależności między komponentami.

Instrukcja uruchamiania aplikacji

Krok 1.
Po pobraniu projektu należy rozpakować archiwum i otworzyć plik .sln w środowisku Visual Studio. Rozwiązanie zawiera wszystkie niezbędne warstwy: domenę, aplikację, infrastrukturę oraz warstwę prezentacji — nie trzeba dodawać niczego ręcznie.

Krok 2.
Przed uruchomieniem aplikacji warto sprawdzić konfigurację połączenia z bazą danych. W pliku appsettings.json znajduje się sekcja ConnectionStrings, a w niej parametr DefaultConnection. Można zostawić domyślną wartość (dla lokalnego SQL Server Express), lub podać własne dane dostępu do instancji SQL Server.
W tym samym pliku znajdują się dane konfiguracyjne SMTP. Domyślnie ustawiony jest serwer SendGrid, działający na konkretnym koncie testowym. Jeśli zachodzi potrzeba, można zastąpić go własnym serwerem SMTP (np. Gmail, Outlook, itp.), podając odpowiedni host, port oraz dane logowania.

Krok 3.
Po ustawieniu konfiguracji należy przejść do *NuGet Package Manager Console* i wykonać komendę:

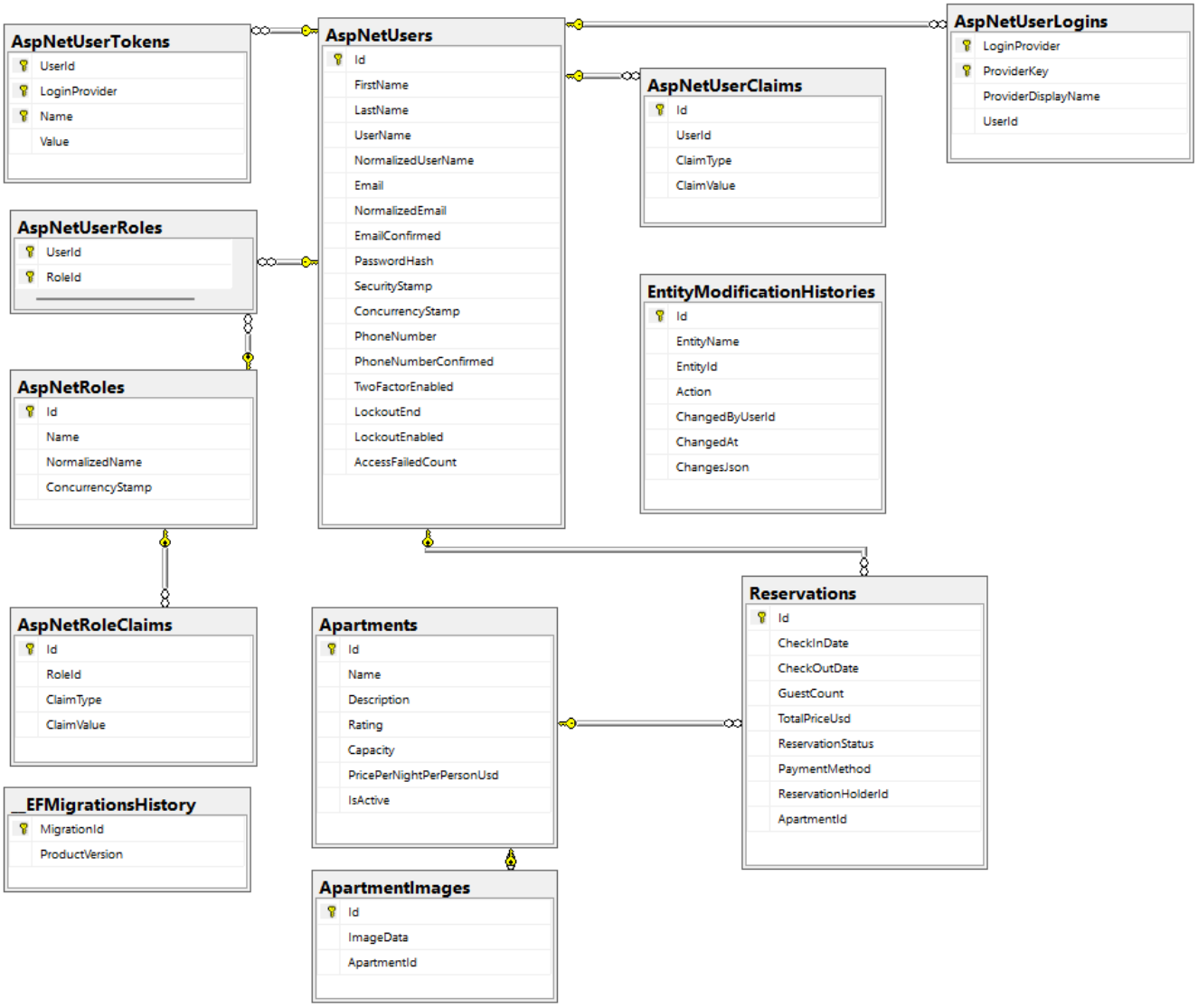
```
Update-Database
```

Polecenie to zastosuje aktualne migracje Entity Framework i utworzy wszystkie wymagane tabele w bazie danych, zgodnie z modelem domenowym.

Krok 4.
Po wykonaniu powyższych kroków aplikacja jest gotowa do uruchomienia. Można ją wystartować bezpośrednio z poziomu Visual Studio.

Krok 5.
Dla otrzymania roli Administratora, treba za pomocą przeglądarki zrobić HTTP GET żądanie na adres account/backdoor/get-admin-role

Opis struktury bazy danych



Struktura bazy danych składa się zarówno z tabel systemowych, generowanych automatycznie przez framework, jak i z tabel odpowiadających modelom biznesowym aplikacji. Dzięki takiemu podziałowi możliwe jest oddzielenie logiki aplikacyjnej od mechanizmów technicznych, takich jak autoryzacja użytkownika czy zarządzanie migracjami.

Tabele systemowe ASP.NET Identity

Wszystkie tabele rozpoczynające się od prefiksu AspNet są częścią wbudowanego systemu *ASP.NET Identity*, który odpowiada za zarządzanie użytkownikami i bezpieczeństwem aplikacji. Tabele te są tworzone automatycznie i służą do przechowywania informacji o kontach użytkowników, rolach, hasłach, tokenach logowania oraz przypisanych uprawnieniach.

Wśród nich warto wyróżnić:

a. AspNetUsers – główna tabela użytkowników, zawierająca dane logowania oraz dodatkowe pola, takie jak imię, nazwisko, e-mail, numer telefonu i ustawienia zabezpieczeń. Tabela ta jest aktywnie wykorzystywana w aplikacji i powiązana z logiką biznesową, m.in. przez relację z tabelą Reservations.
b. AspNetRoles – tabela przechowująca zdefiniowane role użytkowników (np. administrator, klient). Współpracuje z innymi tabelami w celu kontroli dostępu do określonych funkcji systemu.

Pozostałe tabele (AspNetUserRoles, AspNetUserClaims, AspNetUserLogins, AspNetUserTokens, AspNetRoleClaims) pełnią funkcje pomocnicze w zakresie przypisywania uprawnień, autoryzacji zewnętrznej oraz tokenów bezpieczeństwa. Ich obecność jest wymagana przez mechanizm Identity, ale nie są bezpośrednio wykorzystywane w modelach biznesowych.

Tabele systemowe Entity Framework

Tabela _EFMigrationsHistory to tabela techniczna, tworzona automatycznie przez *Entity Framework*. Przechowuje historię zastosowanych migracji w projekcie, w tym identyfikator migracji oraz wersję użytego silnika. Jej obecność pozwala na zachowanie spójności między modelem danych w kodzie a strukturą bazy danych.

Modele biznesowe aplikacji

Wśród modeli dziedzinowych kluczową rolę odgrywają dwie główne tabele: Reservations oraz Apartments. Są one bezpośrednio związane z funkcjonalnością aplikacji i odpowiadają za obsługę procesu rezerwacji apartamentów.

Tabela Apartments reprezentuje jednostki możliwe do wynajęcia. Przechowuje podstawowe informacje o apartamentach, takie jak nazwa, opis, ocena, liczba miejsc, cena za osobę oraz status aktywności. Z apartamentami powiązana jest również tabela ApartmentImages, która umożliwia dodawanie zdjęć. Obrazy są przechowywane w formie binarnej (jako bajty), co pozwala na ich bezpośrednie pobieranie i wyświetlanie w aplikacji.

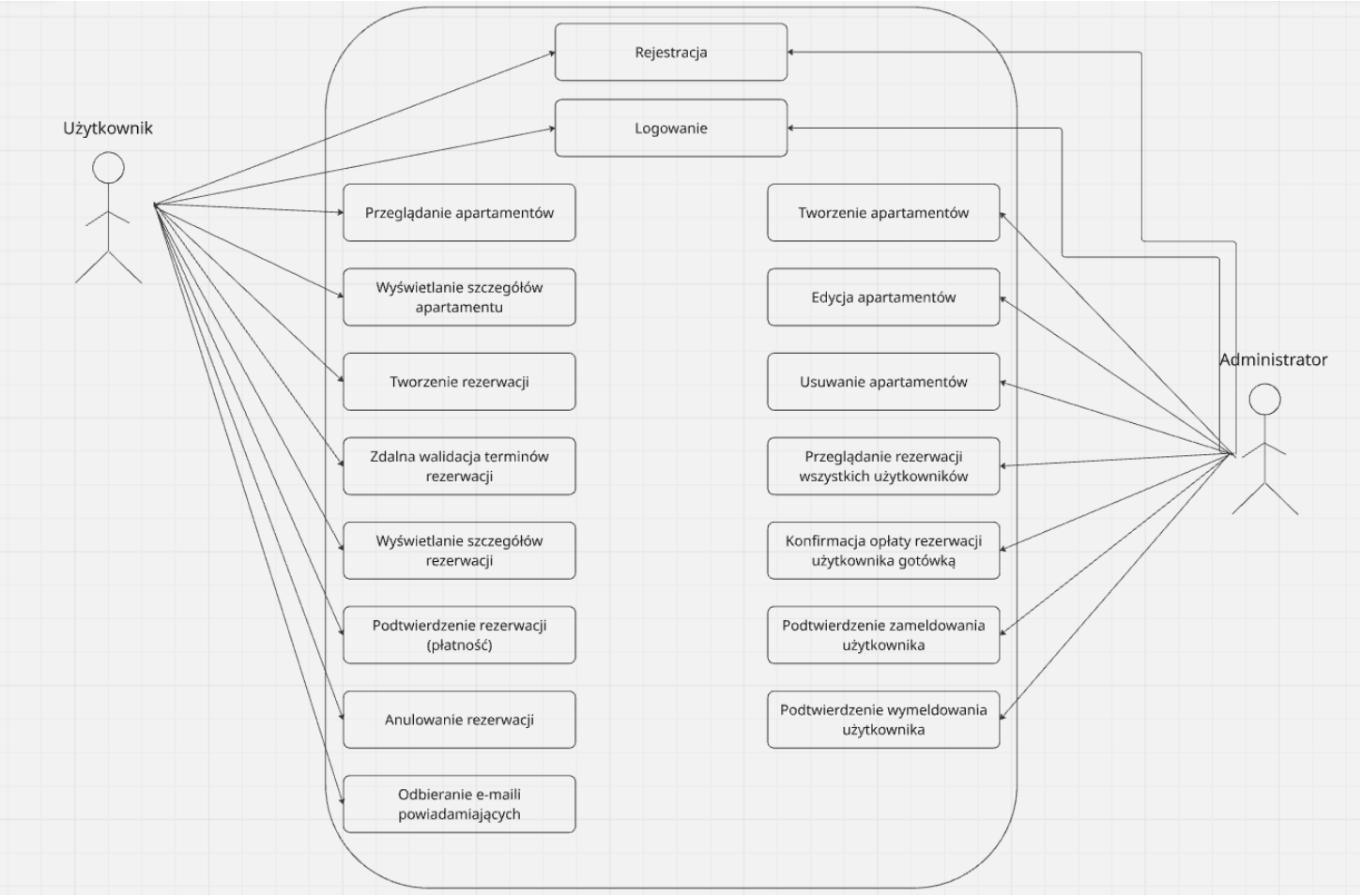
Tabela Reservations zawiera dane dotyczące każdej rezerwacji wykonanej przez użytkownika. Uwzględnia takie informacje jak daty zameldowania i wymeldowania, liczba gości, całkowita cena, metoda płatności, status

rezerwacji, identyfikator jednostki do wynajęcia oraz identyfikator użytkownika dokonującego rezerwacji. Relacja z AspNetUsers pozwala wiązać rezerwacje z konkretnymi kontami w systemie, a relacja z Apartments pozwala wiązać rezerwacje z konkretnymi apartamentami.

Historia zmian encji

Tabela EntityModificationHistories została dodana do projektu w celu śledzenia zmian w encjach aplikacji. Rejestruje informacje o każdej zmianie wykonanej na obiektach domenowych podczas zapisu do bazy danych za pomocą SaveChanges() lub SaveChangesAsync(). Przechowywane są m.in. nazwa encji, identyfikator zmienionego rekordu, rodzaj operacji (dodanie, modyfikacja, usunięcie), użytkownik dokonujący zmiany, czas operacji oraz zawartość zmiany w formacie JSON. Tabela ta umożliwia monitorowanie historii działań użytkowników oraz audyt zmian w kluczowych danych systemu.

Diagram przypadków użycia



Opis kluczowych elementów backendu

Backend aplikacji został zaprojektowany w sposób warstwowy, z wyraźnym podziałem odpowiedzialności pomiędzy kontrolery, serwisy, repozytoria i system obsługi zdarzeń domenowych. Zastosowane rozwiązania wspierają przejrzystość kodu, testowalność oraz możliwość łatwej rozbudowy aplikacji w przyszłości.

Kontrolery

W aplikacji znajduje się pięć głównych kontrolerów, z których dwa odpowiadają za podstawowe funkcje systemu, a trzy realizują kluczowe operacje biznesowe.

AccountController

Odpowiada za rejestrację, logowanie oraz zarządzanie tożsamością użytkownika. Korzysta z funkcji ASP.NET Identity, wspierając autoryzację, uwierzytelnianie i zarządzanie sesją użytkownika. Kontroler udostępnia formularze rejestracyjne oraz logowania, a także odpowiednie akcje obsługujące ich przesyłanie i walidację.

HomeController

Służy wyłącznie do prezentacji strony głównej (landing page). Dane wyświetlane na stronie są zgenerowane, i nie pochodzą z bazy danych. Celem kontrolera jest przedstawienie przykładowych treści w atrakcyjnej formie graficznej – niezależnie od danych aplikacyjnych.

ApartmentController

Jeden z dwóch kluczowych kontrolerów biznesowych. Występuje w dwóch wariantach:

- W wersji publicznej użytkownicy mogą przeglądać dostępne apartamenty, korzystać z opcji filtrowania oraz wyświetlać szczegóły ofert.
- W wersji administracyjnej (obsługiwanej w Area Administrator) użytkownicy z rolą administratora mają dodatkowy dostęp do tworzenia, edytowania i usuwania apartamentów oraz podglądu powiązanych rezerwacji.

System filtrowania apartamentów jest wspólny dla obu wariantów i opiera się na dynamicznym generowaniu zapytań LINQ.

ReservationController
Drugi z głównych kontrolerów domenowych, również dostępny w dwóch wersjach:

- W trybie użytkownika końcowego kontroler umożliwia tworzenie nowych rezerwacji (np. z poziomu widoku apartamentu), przeglądanie własnych rezerwacji (zakładka "My Reservations"), a także zarządzanie nimi – w tym możliwość anulowania rezerwacji oraz opłacenia jej.
  Zastosowano tutaj również mechanizm *Remote Validation*, który służy do sprawdzania dostępności dat na etapie tworzenia rezerwacji. Walidacja odbywa się asynchronicznie z poziomu przeglądarki, za pomocą atrybutu [Remote] i integracji z jQuery Validate.
- W wersji administracyjnej możliwe jest przeglądanie rezerwacji wszystkich użytkowników. Administratorzy mogą również anulować rezerwacje lub przeprowadzać akcje check-in oraz check-out.

PricingController
Ten kontroler udostępnia prosty interfejs API, który pozwala dynamicznie obliczyć cenę rezerwacji na podstawie daty, liczby gości i wybranego apartamentu. Jest wykorzystywany przy tworzeniu rezerwacji.

Serwisy aplikacyjne

Logika biznesowa aplikacji została zorganizowana w dedykowanych serwisach, które izolują ją od kontrolerów oraz operacji dostępowych do danych.

ApartmentService
Zawiera podstawowe operacje CRUD związane z apartamentami. Odpowiada za tworzenie, edycję, usuwanie i pobieranie danych apartamentów. Współpracuje z systemem filtrowania w celu zwracania tylko tych obiektów, które spełniają określone kryteria użytkownika.

ReservationService
Podobnie jak serwis apartamentów, oferuje pełen zestaw operacji CRUD, lecz posiada również bardziej zaawansowane metody specyficzne dla domeny rezerwacji – takie jak CancelReservation, CheckInReservation, CheckOutReservation, ProcessPayment. Serwis uwzględnia walidację dostępności dat oraz statusów rezerwacji.

PricingService
Realizuje jedno zadanie – oblicza całkowity koszt rezerwacji. Wykorzystywany jest w PricingController do dynamicznych kalkulacji oraz wewnętrznie w ReservationService, aby przypisać odpowiednią cenę przy tworzeniu rezerwacji.

EmailSenderService
Odpowiada za wysyłanie wiadomości e-mail, w tym potwierdzeń rezerwacji. Integruje się z zewnętrznym serwerem SMTP (np. SendGrid), a także jest wykorzystywany przez system zdarzeń (dispatcher).

System filtrowania

Aby zapewnić elastyczne i dynamiczne filtrowanie danych, zaprojektowano dedykowane komponenty IQueryBuilder, IFilterRule, QueryOptions. Z tych komponentów korzystają i ich implementują komponenty *ApartmentQueryBuilder oraz ReservationQueryBuilder:*

Oba komponenty przyjmują DTO z wybranymi opcjami filtrowania, a następnie generują dynamiczne zapytania LINQ na podstawie dostarczonych parametrów. Dzięki zastosowaniu biblioteki *LinqKit*, możliwe jest konstruowanie złożonych wyrażeń logicznych (Expression<Func<T, bool>>) w czasie wykonania.

Zwracany jest obiekt QueryOptions, który zawiera:
- wyrażenie filtrujące (FilterExpression),
- wyrażenie sortujące (SortExpression),
- parametry paginacji.

Repozytoria

Repozytoria odpowiadają za bezpośredni dostęp do danych za pośrednictwem Entity Framework Core i korzystają z utworzonych wcześniej QueryOptions. Dzięki temu logika zapytań nie jest osadzona w kontrolerach ani w serwisach, lecz pozostaje czysta i modularna. Repozytoria ApartmentRepository oraz ReservationRepository zawierają metody CRUD i potrafią dynamicznie filtrować dane w oparciu o wcześniej wygenerowane warunki. Także, dla encji rezerwacji na poziome kontekstu bazy danych został dodany query filter, który uniemożliwia otrzymanie rezerwacji innego użytkownika, jeżeli obecny użytkownik nie jest w roli Administratora.

Obsługa zdarzeń

W aplikacji zaimplementowano prosty, ale elastyczny system dystrybucji zdarzeń inspirowany wzorcem *Mediator*. Obiekt typu IDomainEvent przekazywany jest do komponentu DomainEventDispatcher, który wyszukuje wszystkie zarejestrowane handlery obsługujące dany typ zdarzenia i wywołuje ich metody HandleAsync.

System pozwala na podłączenie wielu handlerów do jednego zdarzenia i wykonywanie ich w tle przez BackgroundService. Obecnie handler'y obsługują wysyłkę wiadomości e-mail, ale architektura umożliwia łatwe rozszerzenie o inne działania (np. powiadomienia push, logowanie operacji, synchronizacja z systemami zewnętrznymi).

Obecnie po każdej istotnej akcji nad rezerwacją, jest wysyłany event do DomainEventDispatcher, co powoduje do wysyłki maila do użytkownika.

Walidacja danych

Aplikacja korzysta z wbudowanego systemu walidacji ASP.NET, który pozwala stosować zarówno standardowe atrybuty ([Required], [Range], [EmailAddress]), jak i własne rozszerzenia. Szczególnie istotne jest wsparcie dla *Remote Validation*, które umożliwia wykonywanie walidacji asynchronicznej po stronie serwera z użyciem jQuery.
Dzięki integracji z jQuery Unobtrusive Validation, formularze na froncie automatycznie korzystają z reguł walidacyjnych zdefiniowanych w modelach, co upraszcza utrzymanie spójności pomiędzy backendem a frontendem.

Opis przypadków testowych

Feature: Tworzenie rezerwacji (Użytkownik)

Scenario: Prawidłowe utworzenie rezerwacji
  Given użytkownik wprowadza poprawne dane rezerwacji
  When użytkownik klika "Reserve now"
  Then system tworzy rezerwację i przenosi użytkownika do szczegółów rezerwacji
  And system wysyła wiadomość e-mail z potwierdzeniem rezerwacji

Scenario: Próba stworzenia rezerwacji w zajętym terminie
  Given użytkownik wybiera apartament
  And istnieje już aktywna rezerwacja dla tego apartamentu w tych samych datach
  When użytkownik klika "Reserve now"
  Then system nie pozwala utworzyć rezerwacji
  And użytkownik otrzymuje komunikat o niedostępnych datach
  And rezerwacja nie jest zapisana w bazie
  And żaden e-mail nie jest wysłany

Feature: Zarządzanie rezerwacjami (Użytkownik)

Scenario: Anulowanie aktywnej rezerwacji
  Given użytkownik widzi listę swoich rezerwacji
  When użytkownik klika "Cancel" przy aktywnej rezerwacji
  Then status rezerwacji zmienia się na "Cancelled"
  And system wysyła wiadomość e-mail z potwierdzeniem anulowania

Scenario: Użytkownik próbuje anulować opłaconą rezerwację
  Given użytkownik widzi swoją rezerwację ze statusem "Confirmed"
  When użytkownik klika "Cancel"
  Then system wyświetla błąd.
  And status rezerwacji pozostaje bez zmian
  And żaden e-mail nie jest wysłany

Scenario: Opłacenie rezerwacji
  Given użytkownik ma nieopłaconą rezerwację
  When użytkownik klika na jednu z opcji "Pay with ..."

Then rezerwacja zostaje oznaczona jako opłacona “Confirmed”
And system wysyła wiadomość e-mail z potwierdzeniem płatności

Feature: Zarządzanie rezerwacjami użytkowników (Administrator)

Scenario: Wykonanie check-in w poprawnym terminie
Given administrator widzi rezerwację z datą zameldowania równą dzisiejszej
When administrator klika "Check In"
Then status rezerwacji zmienia się na "Checked In"
And system wysyła wiadomość e-mail potwierdzającą zameldowanie

Scenario: Wykonanie check-out w poprawnym terminie
Given administrator widzi rezerwację z datą wymeldowania równą dzisiejszej
When administrator klika "Check-out"
Then status rezerwacji zmienia się na "Checked Out"
And system wysyła wiadomość e-mail potwierdzającą wymeldowanie

Scenario: Próba wykonania check-in przed datą rozpoczęcia
Given administrator przegląda rezerwację z datą zameldowania w przyszłości
Then przycisk "Check In" jest nieaktywny
And system nie pozwala wykonać tej akcji

Scenario: Próba wykonania check-out przed datą wymeldowania
Given administrator przegląda rezerwację z datą wymeldowania w przyszłości
Then przycisk "Check Out" jest nieaktywny
And system nie pozwala wykonać tej akcji

Scenario: Anulowanie nieopłaconej rezerwacji klienta przez administratora
Given administrator widzi aktywną rezerwację użytkownika
When administrator klika "Cancel"
Then status rezerwacji zmienia się na "Cancelled"
And system wysyła wiadomość e-mail z informacją o anulowaniu

Scenario: Administrator próbuje anulować opłaconą rezerwację
Given administrator widzi rezerwację użytkownika ze statusem "Confirmed"
When administrator klika "Cancel"
Then system wyświetla błąd
And status rezerwacji pozostaje bez zmian
And żaden e-mail nie jest wysłany

Literatura

1. Microsoft Docs: ASP.NET Core MVC — oficjalna dokumentacja frameworka ASP.NET Core, używanego do budowy warstwy backendowej aplikacji.
2. Microsoft Docs: Entity Framework Core — źródło wiedzy na temat używanego ORM-u, sposobu zarządzania migracjami oraz podejścia Code First.
3. Microsoft Docs: ASP.NET Identity — dokumentacja systemu zarządzania tożsamością użytkownika, używanego w aplikacji do logowania i autoryzacji.
4. LinqKit Documentation: https://github.com/scottksmith95/LINQKit — biblioteka używana do dynamicznego budowania zapytań filtrowania danych przy pomocy wyrażeń Expression.
5. SendGrid Docs: https://docs.sendgrid.com — dokumentacja usługi wysyłania wiadomości e-mail, użytej w projekcie do potwierdzeń rezerwacji.
6. Gherkin Syntax: https://cucumber.io/docs/gherkin/ — opis składni języka Gherkin, używanego do definiowania przypadków testowych w formacie Given–When–Then.